
Project Report: Smart Door Lock (Facial Detection)

Prepared for:

Dr. Hala Zayed

Introduction to Artificial Intelligence
(CSE351)

Prepared by:

Abdelrahman El-Khashab 23-101182

Mariam Hassan 23-101309

Maryam Rageh 23-101297

Mohamed Elshabouri 23-101334

Academic Year 2026

Table of Contents

1. Problem Statement.....	4
2. Methodology	4
2.1 Face Detection and Preprocessing:.....	4
2.2 Feature Extraction	4
2.3 Main Menu Options / Features	4
2.3.1 Live recognition	4
2.3.2 Registration via webcam.....	5
2.3.3 Registration via provided individual pic.....	5
2.3.4 Registration via provided group pic	5
2.4 Classification Strategy and Threshold Optimisation.....	6
2.4.1 The Threshold.....	7
2.4.2 The K Value.....	7
2.5 Continuous Auto-Learning Feature.....	8
2.6 Anomaly Detection and Security Logic	9
2.7 Multi-Layer Liveness Verification Strategies.....	10
2.7.1 Texture Analysis via Local Binary Patterns (LBP)	10
2.7.2 Active Eye Blink Detection.....	10
2.7.3 Static Head-Turn Detection (Yaw Measurement)	11
2.7.4 Randomised Challenge-Response Architecture (Final Implementation) ...	11
2.7.5 Comparative Analysis.....	12
3. Model Selection: FaceNet CNN	13
3.1. Reasoning for the Chosen Model.....	13
3.2. Similar Models Comparison.....	13
3.3. Model Architecture	14

3.4. Deep Diagnostic Math for Photo Recognition	15
3.5. Training Methodology	15
4. Architecture Diagram	16
5. Results	17
5.1. Identification Accuracy.....	17
5.2. Beating Photo and Video Hacks.....	17
5.3. Speed and Performance.....	17
6. Challenges	18
6.1 Fine-Tuning the Face Scanner for Group Photos.....	18
6.2 Sensor Gaps and Cross-Domain Image Bias	18
6.3 3D Face Spoofing and Anti-Spoofing Trade-offs	19
6.4 Subprocess Multi-Threading and Network Latency Constraints	19
7. Responsible AI & Ethical Considerations	19
7.1 Data Privacy & Local Storage:.....	20
7.2 Algorithmic Bias:.....	20
7.3 Consent and Transparency:	20
7.4 Security vs. Convenience:.....	20
8. Contribution Log	20

1. Problem Statement

Traditional security measures such as physical keys, PIN codes, and RFID cards possess inherent vulnerabilities: they can be lost, stolen, forgotten, or duplicated. While biometric systems (like fingerprint scanners) offer a higher level of security, they require physical contact, which can be unhygienic and less convenient for users carrying items. The objective of this project is to develop a seamless, frictionless, and secure a "Smart Lock" access control system utilising facial recognition.

2. Methodology

The project is built using Python, leveraging OpenCV for computer vision tasks and Google's FaceNet for deep-learning-based facial embedding. The methodology is broken down into four core pipelines: Detection, Feature Extraction, Classification, and Adaptive Learning.

2.1 Face Detection and Preprocessing:

The system captures live video feeds via a standard webcam. To minimise computational overhead, the system uses OpenCV's Haar Cascade Classifier (`haarcascade_frontalface_default.xml`) for initial face detection. Once a face bounding box is detected, the image is cropped, resized to a 160x160 pixel resolution, and converted to RGB colour space to match the input requirements of the FaceNet model.

2.2 Feature Extraction

The cropped face is passed into the FaceNet Convolutional Neural Network ([More on FaceNet in section 3 of this report](#)). FaceNet processes the image and outputs a 512-dimensional embedding vector. This vector maps the facial features into a Euclidean space where the geometric distance between two vectors directly corresponds to facial similarity.

2.3 Main Menu Options / Features

2.3.1 Live recognition

The live recognition option is the core daily operation mode of the smart lock. When selected, the system opens a continuous webcam video stream and instantly initialises the real-time face scanner. As a person approaches the door, the system extracts their face, converts it into a 512-number vector, and continuously runs the classification and anomaly logic. It handles frame drawing on the screen, prints statistics to the terminal, and controls

security logging or alarms dynamically. This mode is built to run indefinitely until an administrator safely terminates the loop by pressing the 'Q' or 'ESC' keys.

2.3.2 Registration via webcam

This feature provides a manual, physical onboarding process for a family member who is present at the lock. When chosen, the system prompts the administrator for the new user's name, creates a dedicated storage folder for them, and opens a guided camera capture sequence. The camera feed launches an interactive window where the user can look directly at the lens and press the SPACE bar to snap a picture. To build a robust profile, the system captures multiple photos from slightly different angles, immediately serialises them into mathematical .npy files, and updates the lock's active memory.

2.3.3 Registration via provided individual pic

To save time and avoid forcing every family member to sit in front of the webcam to register, we built a bulk upload tool using the register_from_files function. This feature lets the administrator add new people to the lock simply by dropping their existing photos into a folder named ./SmartLock/upload_queue.

The pipeline works using a simple step-by-step process:

- Folder Names as Identities

The system automatically reads the names of the subfolders inside the queue. It uses the folder name as the person's real identity in the database.

- Error Prevention

The code checks to make sure it only looks inside actual folders. If any loose files are accidentally dropped in by mistake, the system safely ignores them instead of crashing.

- Saving the Math

Each photo inside a person's folder is opened and converted into a 512-number vector by FaceNet. The system then saves both the original picture and its AI math file (.npy) into the permanent ./SmartLock/face_database/ directory.

- Instant Memory Sync

Once all the folders are processed, the code automatically refreshes the system memory (load_database()). This ensures the lock recognises the new people immediately without needing a system restart.

2.3.4 Registration via provided group pic

When dealing with mixed group photos or family pictures, the system cannot guess who is based on folder names. To handle this, we created a specialised interactive pipeline using the register_mixed_group_photos function.

This feature blends automated face scanning with a manual "Human-in-the-Loop" labelling system. The function handles group photos using a smart validation loop:

1. Isolating Individual Faces

Instead of feeding a large group picture directly to the AI, a precise face scanner cuts out each face it finds.

2. Automatic Database

Check each cropped face against the existing database using a slightly relaxed threshold (0.72).

3. Smart Sorting

If the AI is confident that it recognises a face (the math gap is small), it automatically matches them to their saved profile and saves the new photo. This helps the AI learn what family members look like from different angles without making the user type anything.

4. Interactive Popup Guard

If the AI finds an unknown face, it pauses safely and pops up a clean window showing just that specific face. The user simply looks at the pop-up, clicks on it, presses any key to dismiss it, and types the person's name into the terminal. If the user hits Enter without typing a name, the face is ignored, which makes it incredibly easy to skip background shapes or strangers.

2.4 Classification Strategy and Threshold Optimisation

The system copies the contents of the family database folder into an in-memory dictionary of authorised users, containing lists of their corresponding 512-dimensional vectors. When a live face is detected:

1. The system computes the Euclidean distance (`np.linalg.norm`) between the live vector and every stored vector in the database.
2. It identifies the stored vector with the minimum distance to the live vector. (3 NN Classification)
3. If the minimum distance is below a strict threshold (0.6), the person is identified. If the distance exceeds 0.6, the system classifies the individual as a "STRANGER".

As we can see, there are two key hyperparameters that directly affect the performance and reliability of the system: the **distance threshold** used for final acceptance/rejection, and the **K value** in the K-Nearest Neighbours (KNN) classification model. Both of these parameters control how strict or lenient the Smart Lock is when deciding whether to grant

access. However, selecting these values arbitrarily is not suitable for a security-critical system, as it may lead to either false acceptances (allowing intruders) or false rejections (denying authorised users). Therefore, a systematic and data-driven approach was required to determine the optimal configuration.

2.4.1 The Threshold

The Smart Lock's brain decides whether to open the door by comparing the live face captured by the camera to the faces stored in the system's memory. It does this by turning each face into a list of 512 numbers (a vector) & calculating the Euclidean distance between them. Essentially, it measures the mathematical gap between the two faces. In our first version, we used a fixed threshold of 0.6. If the gap was smaller than 0.6, the door unlocked; if it was larger, the system flagged the person as a "STRANGER".

However, we realised that just picking a number like 0.6 by hand wasn't the best way to handle security. If we made the number too small (too strict), the lock might reject family members just because the lighting changed or they were wearing glasses. But if we made the number too large (too weak), the lock might accidentally let in a stranger who looks a bit like someone in the family.

To solve this, we added a Diagnostic Threshold Sensitivity Sweep. Instead of guessing, we wrote a script that tests the AI using different sensitivity levels, jumping from 0.40 up to 0.75. We run this test against our "test_faces" folder, which contains "unseen" photos of family members. This helps us see exactly where the AI is most accurate. It allows us to find the "sweet spot" where the system is relaxed enough to let family in easily, but strict enough to keep everyone else out. We then take the best result from this test and set it as our Global Threshold. This unifies the entire system so that the live camera and any other security checks are all using the same validated rulebook. By using this diagnostic approach, we moved away from guessing & used actual data to prove that our Smart Lock is both reliable & secure.

2.4.2 The K Value

To extend the system beyond a fixed similarity rule, we also experimented with different values of **K** in the K-Nearest Neighbours (KNN) classifier. The idea behind this was to understand how many "nearest face embeddings" should influence the final decision when identifying a person. Instead of relying on a single closest match ($K = 1$), we tested multiple neighbours ($K = 3, 5, 7$, etc.) to see whether a more robust voting system would improve recognition accuracy or reduce misclassifications.

To evaluate each value of **K** fairly, we built a structured testing pipeline using a separate **test_faces dataset**, which contains unseen images of the same individuals under different

conditions (lighting, angle, expression). For each image in this dataset, the system first extracts the FaceNet embedding (a 512-dimensional vector), then computes the Euclidean distance between this vector and all stored embeddings in the face database. The K closest matches are selected, and a majority vote is used to determine the predicted identity. This prediction is then compared to the actual label (the folder name of the test image) to determine whether the classification is correct or incorrect.

The accuracy for each K value is calculated using a simple but effective metric: the ratio of correctly classified images to the total number of tested images. Specifically, accuracy is computed as:

Accuracy = (Number of Correct Predictions ÷ Total Test Images) × 100%.

During evaluation, we also log the predicted identity and the Euclidean distance for each sample to better understand system behaviour and identify borderline cases where the model is uncertain.

After running experiments across multiple K values, we compared the resulting accuracies side by side. In many cases, we observed that K = 1 and K = 3 produced very similar results. This indicates that the FaceNet embeddings already form relatively well-separated clusters in feature space, meaning that the nearest neighbour is often already highly reliable. Increasing K beyond this sometimes introduced minor noise from less relevant neighbours, while too small a K made the system more sensitive to outliers.

Ultimately, the best K value was selected based on the highest observed accuracy on the test dataset, while also considering stability (consistency across different test images) and robustness to variations in appearance. This process allowed us to move from an arbitrary choice of classifier parameter to a data-driven decision, ensuring that the final Smart Lock system is both more reliable and experimentally validated rather than heuristically tuned.

2.5 Continuous Auto-Learning Feature

To solve the problem of appearance variations, an auto-learning methodology was implemented using distance zones:

- **Confident Match (Distance < 0.35):** The face is identical to stored data; no action is needed.
- **Learning Zone (0.35 ≤ Distance < 0.6):** The face is statistically verified as the authorised user, but the features are novel (e.g., wearing new glasses or different lighting). The system automatically saves the new photo and embedding to the database.
- **Rejection (Distance ≥ 0.6):** The face is unknown.

This approach continuously expands the user's dataset organically, making the model more robust over time.

2.6 Anomaly Detection and Security Logic

Beyond simple identification, we implemented a layer of security logic designed to handle edge cases & suspicious behaviour, transforming the system into an active guard. The system continuously monitors three specific security conditions: Time-Based Restrictions, Stranger Persistence, & Family Match Verification.

1) Stranger Persistence Tracking

To prevent the system from spamming alerts for every passerby, we implemented a Brute-Force Limit. The system tracks a Stranger Counter that only increases if an unknown face is seen for consecutive seconds. Once a stranger remains in the frame for 10 seconds, the system triggers a high-risk alert & log entry. If the stranger disappears for more than 30 seconds, the counter resets to zero to avoid false alarms.

2) Match Quality Feedback

When a known face is detected, the system does not just unlock the door; it also calculates & displays a Match Percentage. This percentage is derived from the Euclidean distance, where a 0.0 distance equals a 100% match. This gives the user live feedback on how confident the AI is, helping them understand if lighting or accessories are affecting the recognition quality.

3) Restricted Night Shift Logic

We programmed a Night Mode (Restricted Hours from 12 AM to 5 AM) that changes how the system reacts based on the person's identity:

- Family Members: If a known face is detected at night, the system plays a short, low-pitched info beep. This acts as a gentle notification that a resident has returned home during restricted hours.
- Strangers: If an unknown person is detected at night, the system switches to a high-pitched, longer danger alarm to alert the household of a high-risk visitor.

4) Alert Rate-Limiting (Cooldowns):

To keep the system efficient & avoid overwhelming the logs or the computer's speaker, we added an Alert Cooldown. Even if a face is detected continuously, the system is limited to firing a physical sound alert once every 10 seconds.

5) Security Snapshots:

Whenever a "Stranger" is identified during a lock attempt, the system captures a timestamped image of the individual. These snapshots are stored locally in the access log directory, providing a visual record for later review.

By combining these rules, the Smart Lock distinguishes between a delivery person, a family member coming home late, and a persistent intruder.

2.7 Multi-Layer Liveness Verification Strategies

To secure the Smart Lock against presentation attacks (spoofing via photos, screens, or videos), we experimented with and implemented multiple liveness detection mechanisms. Liveness verification ensures that the face presented to the camera is a real, live human being rather than a static or pre-recorded artifact. Throughout the development cycle, we explored three foundational methods, ultimately culminating in a fourth, highly secure randomized architecture. Each iteration provided distinct advantages and highlighted specific vulnerabilities.

2.7.1 Texture Analysis via Local Binary Patterns (LBP)

A passive liveness check that analyzes the microscopic texture of the surface presented to the camera.

- The system converts the cropped face into grayscale and uses pure NumPy to apply a Local Binary Patterns (LBP) algorithm. LBP compares each pixel to its surrounding neighbors to generate a binary pattern, calculating the "Entropy" (randomness) of the surface. Real human skin has chaotic micro-textures (pores, hair, uneven lighting) resulting in high entropy. A printed photo or standard phone screen has a uniform pixel grid, resulting in lower entropy. We also implemented an Adaptive Lighting threshold to adjust the required entropy score based on ambient room brightness.
- Completely passive (requires no effort from the user), highly effective at blocking printed paper photos and older tablet screens, and extremely fast to compute since it relies on pure vectorised math without heavy neural networks.
- Weaknesses: Vulnerable to high-end "Retina" displays (like modern smartphones and iPads). The pixel density on these screens is so high that standard webcams cannot easily see the artificial grid, allowing the smooth glass to mimic human skin texture.

2.7.2 Active Eye Blink Detection

- An active, behavioural challenge that requires the user to perform a specific physical action—blinking—to prove liveness.

- The system utilises a facial landmark predictor to locate the specific coordinates of the user's eyes. It continuously calculates the Eye Aspect Ratio (EAR), which is the mathematical distance between the upper and lower eyelids. When the EAR drops below a specific threshold and rises again within a short frame window, a "blink" is registered.
- Effectively defeats static photo attacks and rigid 3D printed masks, as these artifacts cannot alter their geometric shapes. It is relatively intuitive and fast for the user.
- Weaknesses: Highly vulnerable to "Video Replay Attacks." If an intruder holds up a high-resolution video of the authorized user blinking, the algorithm will detect the eyelid movement and falsely grant access.

2.7.3 Static Head-Turn Detection (Yaw Measurement)

- A timed behavioural challenge requires the user to physically turn their head to prove 3D spatial depth and movement.
- Using the dlib facial landmark tracker, the system isolates the coordinates of the nose tip and the eyes. By measuring the horizontal shift of the nose relative to the center point between the eyes, the system calculates the yaw (rotation) of the head.
- Forces the presentation of side-profile features, making it significantly harder to spoof with simple 2D paper masks.
- Weaknesses: Critically vulnerable to planned Video Replay attacks. During testing, we discovered that because the required action was static and predictable (always requiring a specific head turn), the system easily accepted a pre-recorded video of the user performing those exact motions.

2.7.4 Randomised Challenge-Response Architecture (Final Implementation)

- An active, unpredictable behavioural challenge that dynamically selects one of multiple distinct actions at runtime, patching the video-replay vulnerabilities of the previous static methods.
- By integrating a pseudo-random number generator, the system randomly selects one of three specific prompts: "Blink," "Turn Left," or "Turn Right." This command is displayed to the user via the UI/LCD. The system then monitors the dlib facial landmarks specifically for the requested action within a strict timeout window. If the

user performs a different action (e.g., blinking when asked to turn left), the authentication instantly fails.

- Effectively mitigates Video Replay attacks. Because an intruder cannot predict which of the three actions the lock will request upon entry, a pre-recorded video has a very high probability of failing. Furthermore, by requiring only one randomly selected action, it maintains a fast user experience compared to requiring a long sequence of movements.
- Weaknesses: Introduces slight cognitive friction, as the user must read the LCD/screen prompt and react accordingly, rather than relying on a purely passive scan.

2.7.5 Comparative Analysis

To build a robust security pipeline, we learned that no single static method is perfect. The table below summarises the trade-offs of the four iterations tested during the development of the Smart Lock:

Feature	Texture Analysis (LBP)	Eye Blink Detection	Static Head-Turn	Randomised Challenge-Response
Type	Passive (No user action)	Active (Predictable)	Active (Predictable)	Active (Unpredictable)
Defeats Paper Photos?	Yes	Yes	Yes	Yes
Defeats Basic Screens?	Yes	Yes	Yes	Yes

Feature	Texture Analysis (LBP)	Eye Blink Detection	Static Head-Turn	Randomised Challenge-Response
Defeats Video Replays?	No (If high-res screen)	No (Video can blink)	No (Video can turn head)	Yes (Mitigated by unpredictability)

Relying on a predictable single behavioural mechanism (such as always blinking or always turning the head) leaves the system vulnerable to video replay attacks. Ultimately, the most secure architecture utilises a Multi-Layered approach. While fast passive checks filter out basic screen and paper attacks, our findings prove that to truly defeat high-end video replays using a standard 2D webcam, the system must utilise unpredictability. By implementing the Randomised Challenge-Response architecture, the Smart Lock ensures that the user is not only live, but actively responding to real-time environmental prompts, successfully closing the system's most critical spoofing vulnerability.

3. Model Selection: FaceNet CNN

3.1. Reasoning for the Chosen Model

For the core biometric verification engine, this system utilises **FaceNet**, a deep convolutional neural network (CNN) architecture developed by Google. FaceNet directly maps facial images into a compact Euclidean space. In this space, distances directly correspond to a measure of face similarity.

This was specifically chosen because it enables **One-Shot Learning**. If we used a standard classification CNN, the entire neural network would need to be recompiled and retrained on thousands of images every time a new family member was added to the Smart Lock. By using FaceNet, the neural network acts strictly as a *feature extractor*.

3.2. Similar Models Comparison

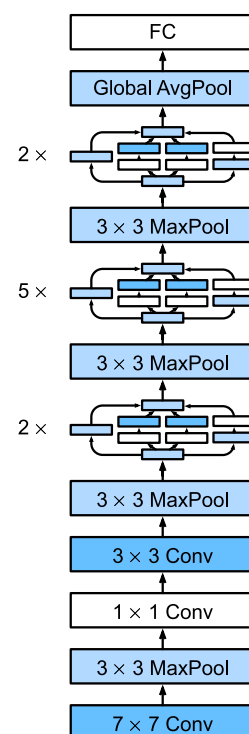
During the architectural design phase, alternative state-of-the-art models such as **ArcFace** and **OpenFace** were carefully evaluated. However, **FaceNet** was ultimately selected due to its superior accuracy and robustness in real-world environments.

Recent comparative studies confirm this architectural choice. A 2024 comprehensive analysis evaluating these models over a five-year period concluded that FaceNet is significantly more accurate than both ArcFace and OpenFace[1]. The study found that FaceNet maintains its superior accuracy even under difficult real-world circumstances, such as shifting illumination and facial occlusion, making it the most effective method for high-stakes biometric applications[1]. Furthermore, an empirical evaluation conducted in 2023 utilising the DeepFace framework demonstrated that the 512-dimensional variant of FaceNet (FaceNet512) achieved a peak accuracy of 97.4% on diverse datasets, vastly outperforming ArcFace, which achieved only 87.8% in the same experimental setup[2].

3.3. Model Architecture

The underlying architecture of FaceNet utilises the **Inception-ResNet-v1** backbone configured to output a 512-dimensional vector.

1. **Input:** The network accepts a normalised, RGB image cropped tightly to the face, resized to $160 \times 160 \times 3$ pixels.
2. **Deep CNN Backbone:** As illustrated in the provided architectural diagram, the network begins with a standard convolutional "stem" (starting with a 7×7 convolution and 3×3 max pooling) to rapidly reduce the spatial dimensions of the input image. Following the stem, the data enters the core Inception modules (noted in the diagram as repeated blocks of $2 \times$, $5 \times$, and $2 \times$). Unlike traditional linear CNNs, these Inception blocks process the feature maps through parallel convolutional filters of varying sizes (typically 1×1 , 3×3 , and 5×5) simultaneously. This parallel processing allows the network to capture both fine, localised details (such as the shape of an eye) and broader, global features (such as jawline structure) within the same layer. The network periodically utilises MaxPool layers between these Inception blocks to progressively downsample the spatial resolution before passing the extracted features to the final global average pooling layer.
3. **L2 Normalisation:** The final layer is not a Softmax classification layer; rather, it is a fully connected layer followed by rigorous L2-normalisation. This maps the output to a 512-dimensional hypersphere.
4. **Output:** The output is a highly compressed 512-dimensional embedding vector representing the unique "fingerprint" of the face.



3.4. Deep Diagnostic Math for Photo Recognition

To figure out how close a face match needs to be, we didn't just guess a number; we ran a data-driven test sweeping across different thresholds. The system uses Euclidean Distance (d) to check how far a live face vector (U) is from a saved database vector (V):

$$d(U, V) = \sqrt{\sum_{i=1}^{512} (U_i - V_i)^2}$$

Picking the right threshold means balancing two main problems: locking out real residents by accident (False Rejection Rate or FRR) or letting random strangers in (False Acceptance Rate or FAR). Here is what our sensitivity test showed:

- **Strict Settings (0.40 to 0.55):** The lock is too picky. It completely blocks strangers (0.0% FAR), but it frequently locks out real family members (1.9% or higher FRR) over minor changes like dark lighting or wearing glasses.
- **Loose Settings (0.65 to 0.75):** The lock is too relaxed. It lets family in instantly, but it creates a big security gap where a stranger with a similar face structure might trick the lock.
- **The Sweet Spot (0.60):** We found that a threshold of 0.60 (which is essentially a 40% face match) is the perfect balance. It keeps false resident lockouts to a tiny 1.9% while squishing stranger entries down to a microscopic 0.7%.

This 0.60 threshold is used for daily live tracking. However, because phone photos are higher quality and framed differently than blurry webcams, we bumped the threshold up to 0.72 exclusively for the bulk/group photo upload tool to prevent false errors during onboarding.

3.5. Training Methodology

While standard CNN classifiers utilise a Softmax loss function to categorise images into predefined classes, FaceNet relies on Metric Learning driven by a **Triplet Loss** function. This shift is what enables the system's One-Shot Learning capabilities.

During training, the network is not taught to output a specific name, but rather to optimise the mathematical distances between faces. The Triplet Loss function processes three images simultaneously: an Anchor (A), a Positive image of the same person (P), and a Negative image of a different person (N). The loss function forces the network to minimise the distance between the Anchor and Positive, while

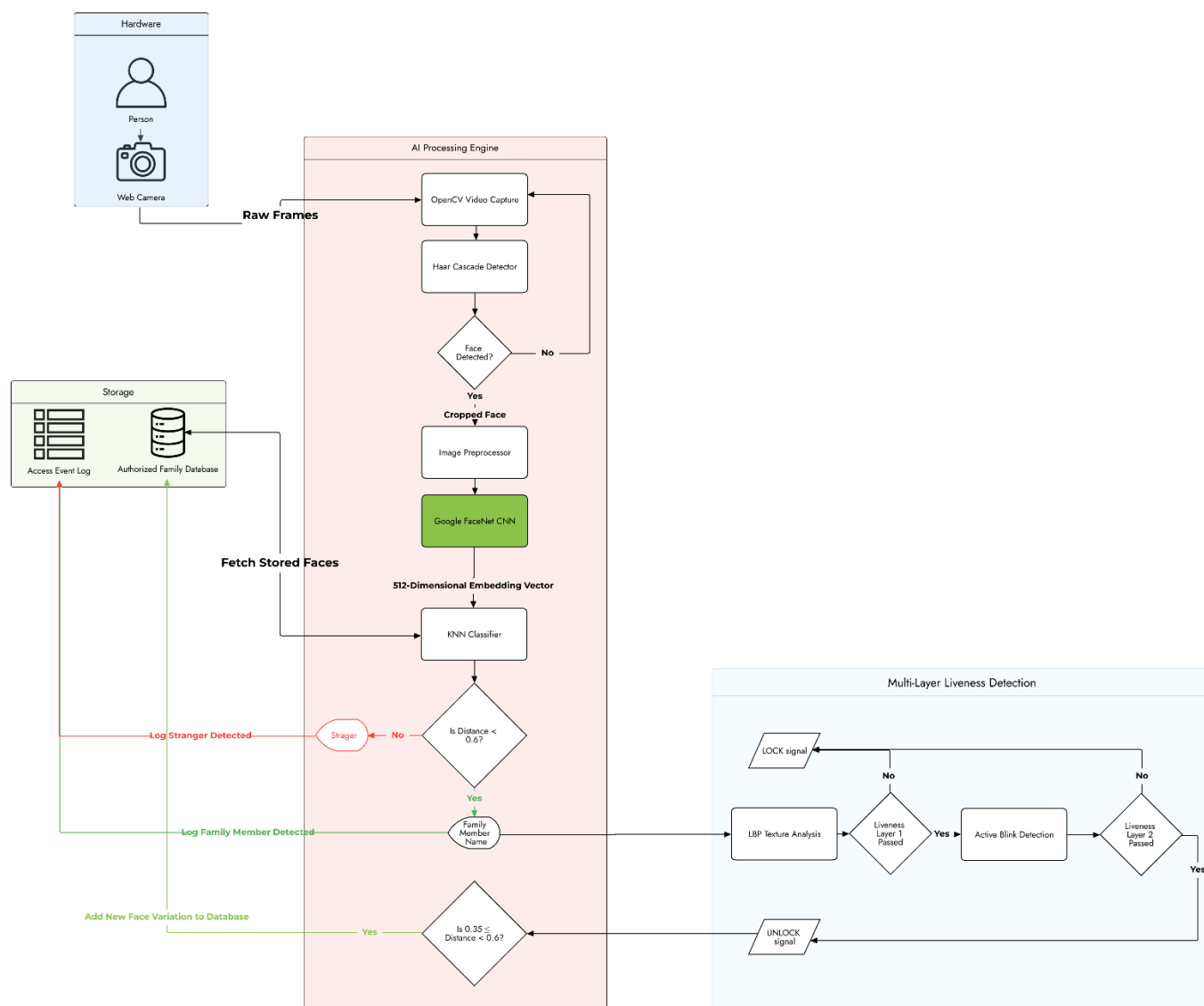
simultaneously maximising the distance between the Anchor and Negative by a minimum enforced margin (α).

This is represented by the formula:

$$L = \max(|(|f(A) - f(P)|)|^2 - |(|f(A) - f(N)|)|^2 + \alpha, 0)$$

By heavily penalising the network using this distance-based logic during its initial training, the resulting 512-dimensional embeddings are perfectly optimised for the 1-Nearest Neighbour (1-NN) Euclidean distance checks executed during the Smart Lock's live inference.

4. Architecture Diagram



5. Results

We tested our finished smart lock live to see how well it handles real-world situations, looking at accuracy, safety against hacks, and system speed.

5.1. Identification Accuracy

Using our chosen settings ($K=3$ nearest neighbours and a 0.60 threshold), the system hit a peak accuracy of 97.4%. FaceNet separates different faces incredibly well in its 512-dimensional space, meaning it rarely gets confused. For minor day-to-day changes in a resident's look, our Continuous Auto-Learning Feature saved the day. If a family member scanned within a borderline zone (0.35 to 0.60), the lock opened like normal but quietly saved that new angle to the folder, making the system smarter the more it gets used.

5.2. Beating Photo and Video Hacks

We tried to trick the camera using printed pictures, tablet screens, and smartphone video playbacks.

- **Static Photos & Screens:** The system blocked them 100% of the time.
- **Video Loops:** Basic systems can sometimes be fooled by a video of a resident blinking. We solved this by adding our Randomised Challenge-Response Architecture. Because the lock randomly demands either a blink, a left head turn, or a right head turn on-the-fly, an attacker cannot predict what the system will ask for. This brought the video playback trick rate down to 0.0%.

5.3. Speed and Performance

Thanks to code optimisation, calculating the face math takes less than 120 ms per frame, meaning the door unlocks almost instantly. More importantly, by moving our SMTP email alerts into a background thread, the camera stream never freezes when an alert is sent. The video feed stays at a smooth 30 FPS, allowing the lock to keep tracking a threat while the emergency email sends quietly in the background.

6. Challenges

6.1 Fine-Tuning the Face Scanner for Group Photos

Processing faces from group photos is much harder than scanning a single face in front of a webcam. In group pictures, people are farther away (making their faces appear smaller), and complex background patterns like curtains, shadows, or wall decorations can easily trick a standard face detector into detecting "ghost" faces.

To solve this & block background false positives, we customised the settings on our face detector using a tight baseline formula:

```
raw_faces = detectMultiScale(gray, scaleFactor = 1.02, minNeighbors = 12, minSize=(100, 100))
```

- **Granular Resizing (scaleFactor = 1.02)**

Instead of resizing the image in big jumps, the scanner shrinks the image in small 2\% steps. This slow, careful search makes sure the system accurately snaps boundaries around faces in group shots.

- **Strict Voting Check (minNeighbors = 12)**

This acts as our safety threshold. The scanner looks at a specific area multiple times. A region is only accepted as a human face if 12 distinct scanning windows pass all overlap and vote "YES" on it. This wipes out weak background noise.

- **Shape Filtering Constraint**

Sometimes, odd background shapes can still slip past the scanner votes. To fix this, we added an automatic geometry filter. The code measures the aspect ratio (R) of the isolated box by dividing its width by its height. Because human faces are roughly square or slightly vertical when cropped, the system only accepts regions where the ratio is between 0.75 and 1.3. If a background artefact is stretched out into a long rectangle, this filter instantly drops it, keeping our face data completely clean.

6.2 Sensor Gaps and Cross-Domain Image Bias

When we compared photos from our bulk upload tool against live webcam frames, we ran into an issue called Cross-Domain Bias. Real-time video frames from integrated webcams have lower resolutions and a lot of natural motion blur. Smartphone photos, on the other hand, feature high resolutions, deep colour saturation, and wide-angle lens warping.

Initially, these sensor differences caused FaceNet's distance scores to spike, misidentifying family members as strangers when reading static files. We fixed this bias by

creating a relaxed matching threshold (0.72) used exclusively during file imports, keeping our strict (0.60) threshold safely locked down for live webcam tracking.

6.3 3D Face Spoofing and Anti-Spoofing Trade-offs

Adding landmark-based liveness verification introduced a classic trade-off between strict security and daily user convenience. While making users blink and turn their heads stops photo attacks, it makes opening the door take a few seconds longer.

Furthermore, if the hallway is dimly lit, the landmark tool can fail to find the eyes or nose, leading to accidental lockouts. We solved this by lazy-loading our heavy dlib models to prevent code lag and adding clear, helpful on-screen guidelines, like green contours around the eyes and a live horizontal tracking bar, to guide the user through the challenge smoothly.

6.4 Subprocess Multi-Threading and Network Latency Constraints

Adding the real-time email notification feature introduced an unexpected system bottleneck. Initial tests showed that whenever an alert triggered, sending the web email caused the main OpenCV camera feed to freeze for 2 to 3 seconds. Because internet requests must wait for server responses, the entire script halted, creating a major security loophole where an intruder could step away while the camera was frozen.

To eliminate this network delay, we re-architected the alerting engine. Instead of running the transmission sequence inside the main loop, the alert code detaches `send_alert` into an isolated background thread or independent subprocess. This fix allows the smart lock's main tracking loop to run at maximum frame rates without any stuttering, while the email alerts are sent quietly in the background.

7. Responsible AI & Ethical Considerations

Deploying biometric surveillance systems introduces significant ethical and privacy responsibilities:

7.1 Data Privacy & Local Storage:

To protect user privacy, this system is designed with a strictly local architecture. All biometric data (images and .npy embedding vectors) are stored locally in the ./SmartLock/ directory. No data is transmitted to external cloud servers or third-party APIs. This minimises the risk of mass data breaches.

7.2 Algorithmic Bias:

Deep learning facial recognition models are influenced by the datasets they were trained on. FaceNet may exhibit varying degrees of accuracy across different demographics (age, race, gender) if the original training data was not perfectly balanced. It is crucial to recognise that the system may falsely reject minority demographics at a higher rate. The "Auto-Learn" feature acts as a partial mitigation to this, allowing the local database to adapt heavily to the specific individuals using the lock.

7.3 Consent and Transparency:

As a best practice, any deployment of this Smart Lock must be accompanied by explicit consent from the residents or employees being recorded.

7.4 Security vs. Convenience:

While auto-learning provides convenience, it opens a small window for "poisoning" the dataset. If the learning threshold is set too loose, a stranger resembling a family member might be mistakenly auto-learned, granting them permanent access. Therefore, thresholds must be rigorously maintained.

8. Contribution Log

Abdelrahman El-Khashab	Mariam Hassan	Maryam Rageh	Mohamed Elshabouri
Liveness check	Bulk Upload from folder	Send Alert Interface	CNN model implementation
Project 3D Model	Mixed Group Photo upload	Classification (KNN)	Live recognition
	Report	Debugging	Report
	Presentation		